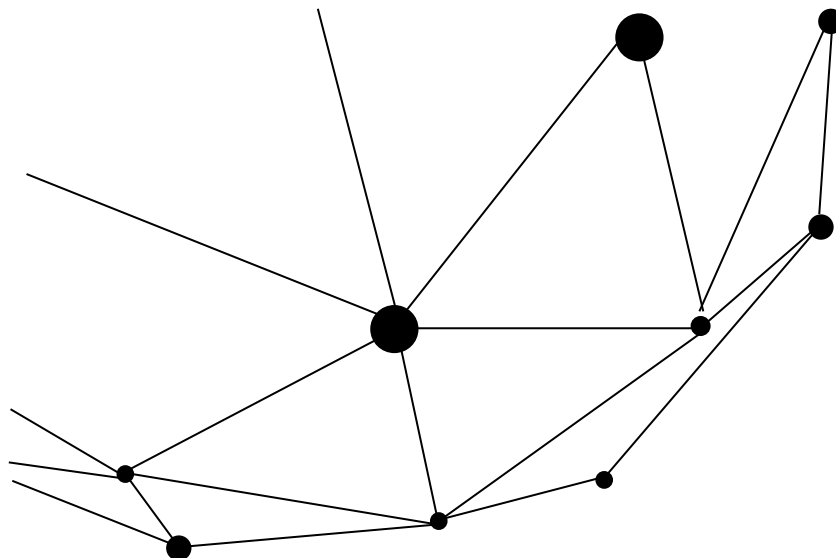




ATLAS

Innovación al alcance de tus manos

MANUAL TECNICO

SISTEMA ATLAS COMPANY 2025

ATLAS.INOVATIONCOMPANY@GMAIL.COM

ATLAS

Innovación al alcance de tus manos

Contenido

Tabla de contenido

1. ARQUITECTURA DEL SISTEMA	4
1.1 Arquitectura General:	4
1.2 Componentes Principales	4
2. TECNOLOGÍAS UTILIZADAS	5
2.1 Frontend Stack	5
2.2 Backend Stack	6
2.3 Base de Datos y Servicios	6
3. ESTRUCTURA DEL PROYECTO	7
3.1 Frontend (/frontend)	7
3.2 Backend (/backend)	11
4. CONFIGURACIÓN DE ENTORNO	14
4.1 Frontend Setup	14
4.2 Backend Setup	15
5. MODELO DE DATOS	16
5.1 Entidades Principales	17
5.1.1 Usuario	17
5.1.2 Proyecto	18
5.1.3 Cliente	18
5.1.4 Tareas	19
5.1.5 ColaboradorProyecto	19
5.1.6 Equipo	19
5.1.7 Agendamiento	20
5.1.8 EquipoAgendamiento	20
5.2 Diccionario de Datos	21
6. APIs y Endpoints	23
6.1 Endpoints Criticos	23

ATLAS

Innovación al alcance de tus manos

7.	AUTENTICACIÓN Y SEGURIDAD	24
7.1	Flujo de Autenticación	24
7.2	Sistema de Roles	24
7.3	Validación de Acceso	24
8.	PATRONES DE DESARROLLO	25
8.1	Patron de Gestión de Información	25
8.2	Ejemplo de Patrón de Cache:	25
9.	SOLUCIÓN DE PROBLEMAS	26
9.1	Problemas Comunes	26
9.2	Logs de Debugging	26
	ARCHIVOS DE REFERENCIA CLAVE	27
	CONTACTO TECNICO	27



ATLAS

Innovación al alcance de tus manos

1. ARQUITECTURA DEL SISTEMA

ATLAS SOFT es un sistema de gestión para microempresas de cómputo que permite administrar proyectos, colaboradores, clientes e inventario de equipos de manera integrada y eficiente.

1.1 Arquitectura General:

El sistema ATLAS SOFT utiliza una arquitectura de **Frontend-Backend separados**:

- **Frontend:** React + Vite (SPA)
- **Backend:** Django REST Framework
- **Base de Datos:** Supabase (PostgreSQL)
- **Autenticación:** Supabase Auth
- **Storage:** Supabase Storage

Frontend SPA
(React / Vite)

Backend API
(Django/DRF)

Supabase DB
(PostgreSQL)

Zustand
(State Mgmt)

Email SMTP
(Invitaciones)

File Storage
(Imágenes)

1.2 Componentes Principales

- **Frontend SPA:** React + Vite para interfaz de usuario
- **Backend API:** Django REST Framework para APIs RESTful
- **Base de Datos:** Supabase PostgreSQL para el respaldo
- **Autenticación:** Supabase Auth + Validación de roles en Django
- **Storage:** Supabase Storage para archivos e imágenes

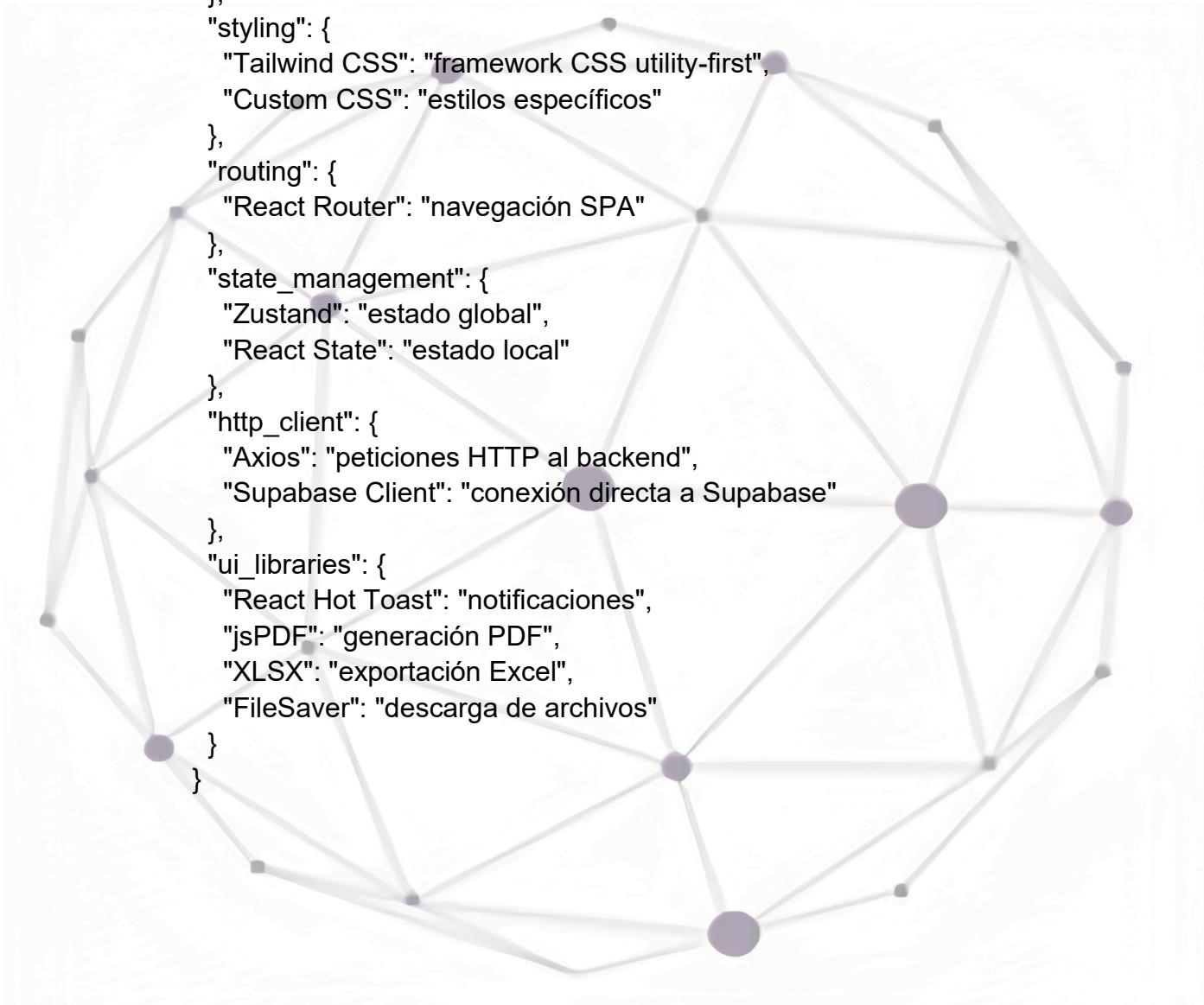
ATLAS

Innovación al alcance de tus manos

2. TECNOLOGÍAS UTILIZADAS

2.1 Frontend Stack

```
{
  "core": {
    "React": "18+",
    "Vite": "bundler y dev server",
    "JavaScript": "ES6+"
  },
  "styling": {
    "Tailwind CSS": "framework CSS utility-first",
    "Custom CSS": "estilos específicos"
  },
  "routing": {
    "React Router": "navegación SPA"
  },
  "state_management": {
    "Zustand": "estado global",
    "React State": "estado local"
  },
  "http_client": {
    "Axios": "peticiones HTTP al backend",
    "Supabase Client": "conexión directa a Supabase"
  },
  "ui_libraries": {
    "React Hot Toast": "notificaciones",
    "jsPDF": "generación PDF",
    "XLSX": "exportación Excel",
    "FileSaver": "descarga de archivos"
  }
}
```



ATLAS

Innovación al alcance de tus manos

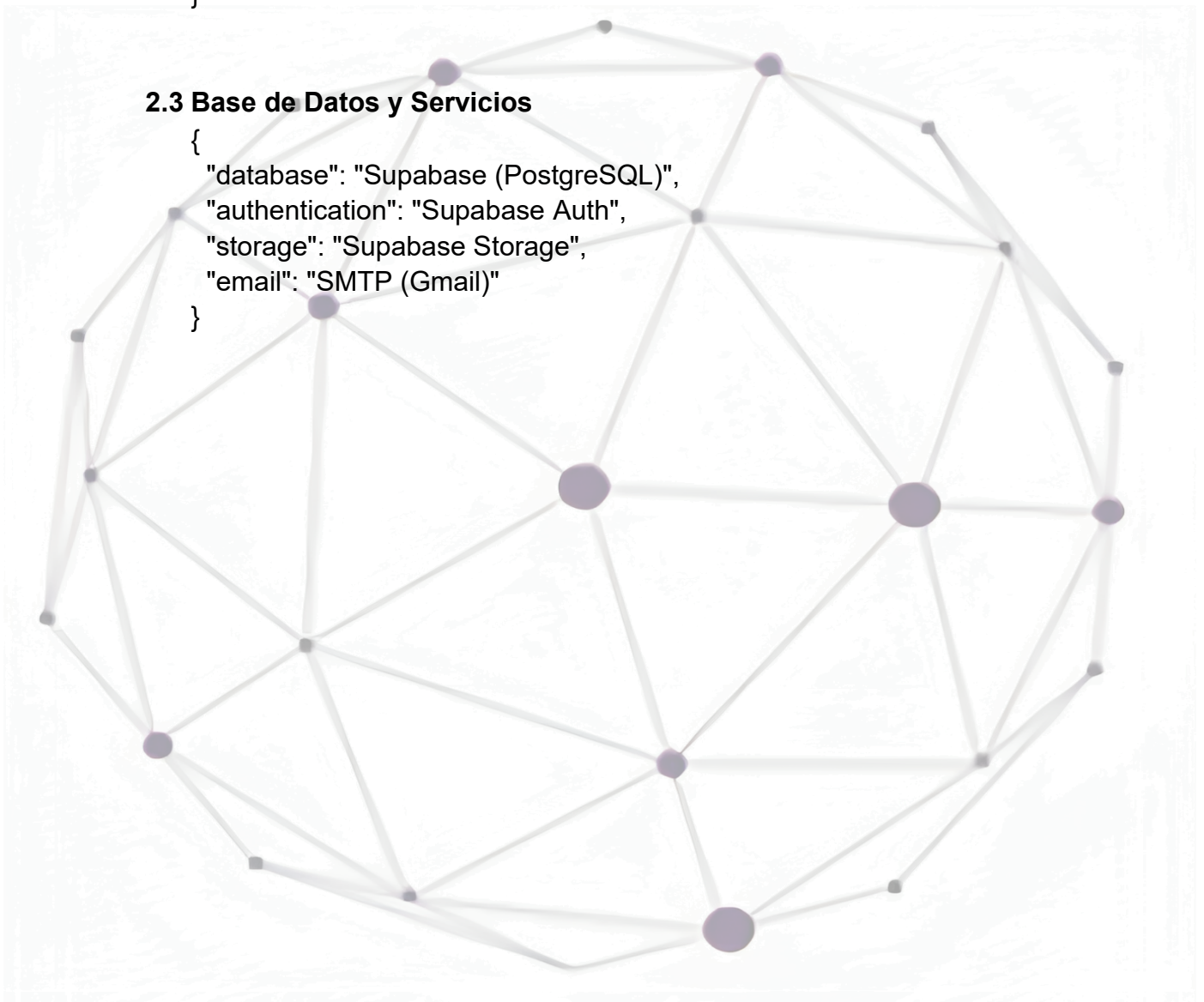
2.2 Backend Stack

Backend principal

```
{  
  "Django": "4.x",  
  "Django REST Framework": "APIs REST",  
  "django-cors-headers": "CORS handling",  
  "Python": "3.8+"  
}
```

2.3 Base de Datos y Servicios

```
{  
  "database": "Supabase (PostgreSQL)",  
  "authentication": "Supabase Auth",  
  "storage": "Supabase Storage",  
  "email": "SMTP (Gmail)"  
}
```

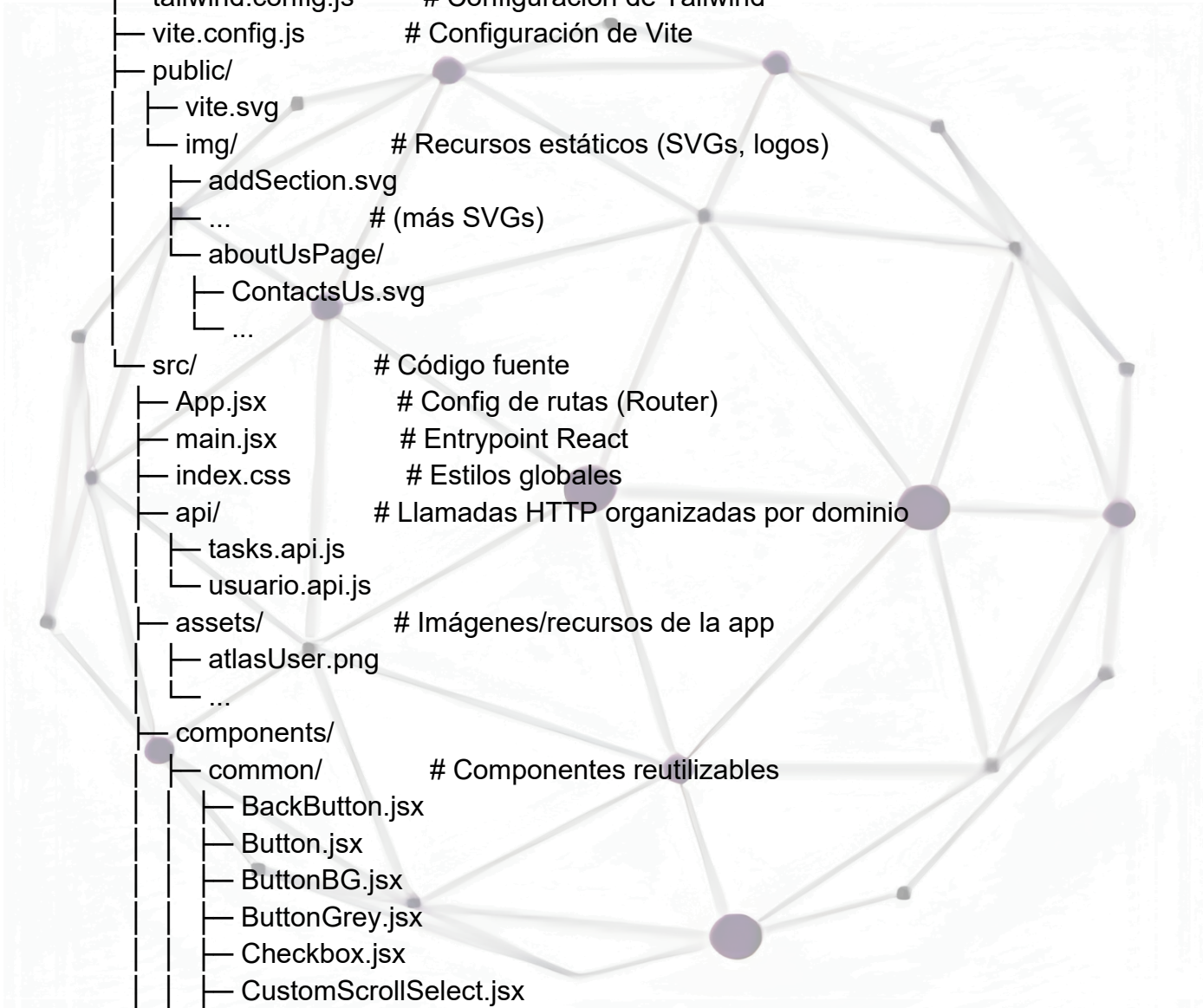


ATLAS

Innovación al alcance de tus manos

3. ESTRUCTURA DEL PROYECTO

3.1 Frontend (/frontend)



```
frontend/
├── .env.local
├── index.html
├── package.json
├── tailwind.config.js
├── vite.config.js
├── public/
│   ├── vite.svg
│   └── img/
│       ├── addSection.svg
│       ├── ...
│       └── aboutUsPage/
│           ├── ContactsUs.svg
│           └── ...
├── src/
│   ├── App.jsx
│   ├── main.jsx
│   ├── index.css
│   ├── api/
│   │   ├── tasks.api.js
│   │   └── usuario.api.js
│   ├── assets/
│   │   ├── atlasUser.png
│   │   └── ...
│   ├── components/
│   │   ├── common/
│   │   │   ├── BackButton.jsx
│   │   │   ├── Button.jsx
│   │   │   ├── ButtonBG.jsx
│   │   │   ├── ButtonGrey.jsx
│   │   │   ├── Checkbox.jsx
│   │   │   ├── CustomScrollSelect.jsx
│   │   │   ├── DataTable.jsx
│   │   │   └── DropdownMenu.jsx
│   │   └── ...
│   └── ...
└── ...
```

Raíz del frontend

Variables locales (VITE_*: URLs, claves públicas)

HTML base de Vite

Dependencias y scripts

Configuración de Tailwind

Configuración de Vite

Recursos estáticos (SVGs, logos)

(más SVGs)

Código fuente

Config de rutas (Router)

Entrypoint React

Estilos globales

Llamadas HTTP organizadas por dominio

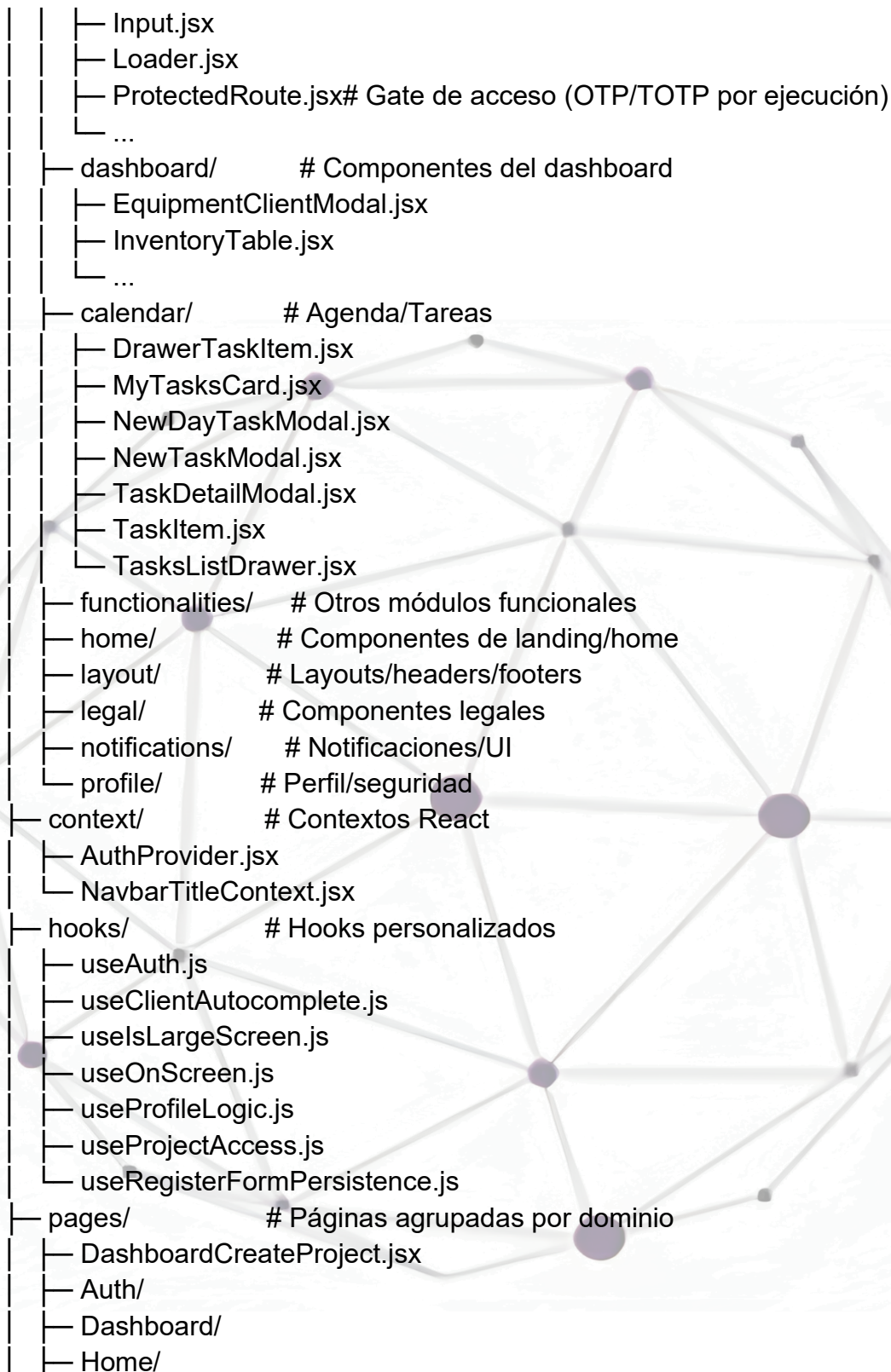
Imágenes/recursos de la app

Componentes reutilizables

Tabla reutilizable (responsive + acciones)

ATLAS

Innovación al alcance de tus manos



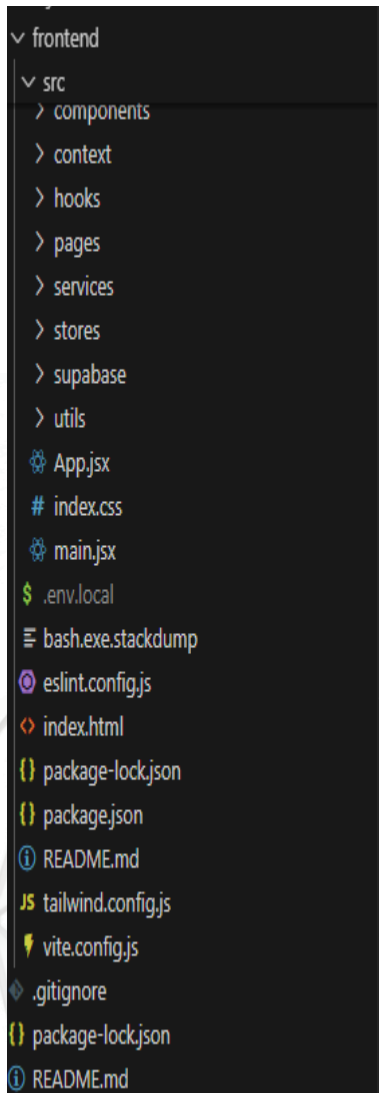
ATLAS

Innovación al alcance de tus manos

- ├─ Legal/
- ├─ Profile/
- ├─ services/ # Servicios (Supabase/Auth/Usuarios)
 - ├─ authService.js
 - ├─ userService.js
- ├─ stores/ # Estado global con Zustand
 - ├─ useClientsStore.js
 - ├─ useCollaboratorsStore.js
 - ├─ useEquipmentsStore.js
 - ├─ useInvitationsStore.js
 - ├─ useProjectsStore.js
 - ├─ useUserStore.js
- ├─ supabase/
- ├─ client.js # Cliente Supabase
- ├─ utils/ # Utilidades
 - ├─ dateUtils.js
 - ├─ fetchAndStoreUser.js
 - ├─ holidayUtils.js
 - ├─ inviteBlocksUtils.js
 - ├─ openDashboardIfActive.js
 - ├─ validateClientForm.js

ATLAS

Innovación al alcance de tus manos



ATLAS

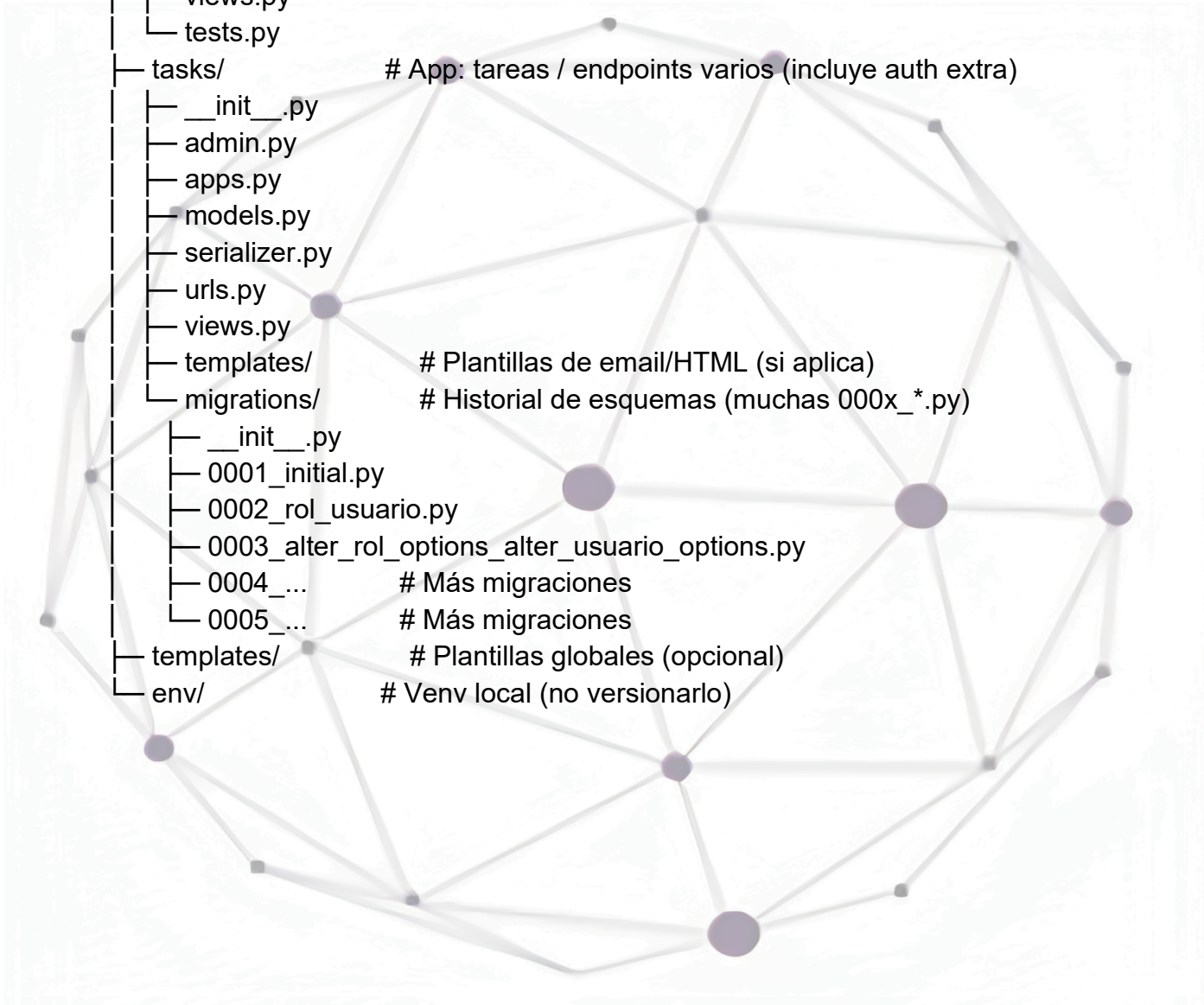
innovación al alcance de tus manos

3.2 Backend (/backend)

backend/	# Raíz del backend
├─ manage.py	# Comandos de Django (runserver, migrate, etc.)
├─ requirements.txt	# Dependencias del backend
├─ modelos.py	# Auxiliar (si no se usa, se puede eliminar)
├─ backend/	# Proyecto Django
│ └─ __init__.py	
│ └─ settings.py	# Config (DB, Apps, CORS, Email, etc.)
│ └─ urls.py	# Rutas principales del proyecto
│ └─ wsgi.py	# Entrypoint WSGI (deploy)
│ └─ asgi.py	# Entrypoint ASGI (websockets)
├─ calendario/	# App: calendario / eventos
│ └─ __init__.py	
│ └─ admin.py	
│ └─ apps.py	
│ └─ models.py	
│ └─ views.py	
│ └─ tests.py	
│ └─ migrations/	
│ └─ __init__.py	
├─ colaboradores/	# App: colaboradores / usuarios internos
│ └─ __init__.py	
│ └─ admin.py	
│ └─ apps.py	
│ └─ models.py	
│ └─ views.py	
│ └─ tests.py	
│ └─ migrations/	
│ └─ __init__.py	
├─ inventario/	# App: inventario de equipos
│ └─ __init__.py	
│ └─ admin.py	
│ └─ apps.py	
│ └─ models.py	
│ └─ serializer.py	
│ └─ views.py	
│ └─ tests.py	
│ └─ migrations/	

ATLAS

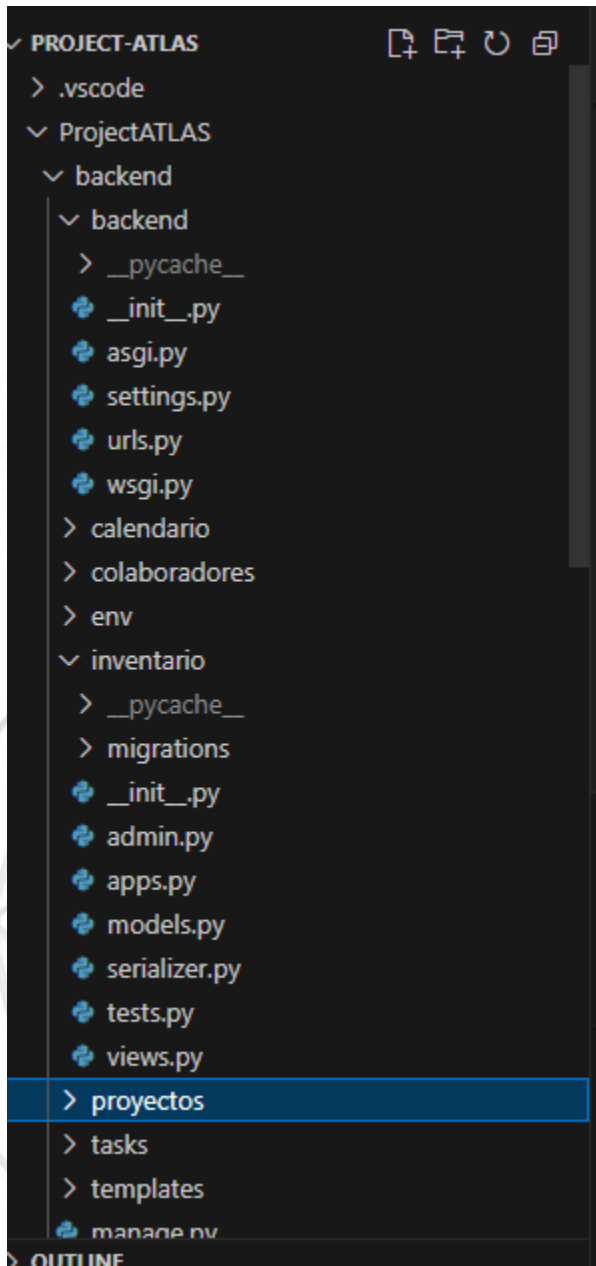
innovación al alcance de tus manos



```
├── __init__.py
├── 0001_initial.py
├── proyectos/                # App: proyectos
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── models.py
│   ├── serializer.py
│   ├── views.py
│   └── tests.py
├── tasks/                    # App: tareas / endpoints varios (incluye auth extra)
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── models.py
│   ├── serializer.py
│   ├── urls.py
│   ├── views.py
│   ├── templates/           # Plantillas de email/HTML (si aplica)
│   └── migrations/          # Historial de esquemas (muchas 000x_*.py)
│       ├── __init__.py
│       ├── 0001_initial.py
│       ├── 0002_roI_usuario.py
│       ├── 0003_alter_roI_options_alter_usuario_options.py
│       ├── 0004_...          # Más migraciones
│       └── 0005_...          # Más migraciones
├── templates/                # Plantillas globales (opcional)
└── env/                      # Venv local (no versionarlo)
```

ATLAS

Innovación al alcance de tus manos



4. CONFIGURACIÓN DE ENTORNO

4.1 Frontend Setup

- **Variables de entorno (.env.local):**

VITE_SUPABASE_URL=https://tu-proyecto.supabase.co

VITE_SUPABASE_ANON_KEY=tu_supabase_anon_key

- **Instalación y Ejecución :**

cd frontend

npm install

npm run dev # Puerto 5173

- **Dependencias Principales (package.json):**

```
{  
  "dependencies": {  
    "react": "^18.0.0",  
    "react-dom": "^18.0.0",  
    "react-router-dom": "navegación",  
    "zustand": "estado global",  
    "axios": "HTTP client",  
    "@supabase/supabase-js": "Supabase client",  
    "tailwindcss": "CSS framework",  
    "react-hot-toast": "notificaciones",  
    "jspdf": "PDF generation",  
    "xlsx": "Excel export"  
  }  
}
```

ATLAS

Innovación al alcance de tus manos

4.2 Backend Setup

- **Configuración Django (settings.py):**

Configuración CORS

```
CORS_ALLOWED_ORIGINS = [  
    "http://localhost:5173",  
    "http://127.0.0.1:5173",  
]
```

```
CORS_ALLOW_ALL_ORIGINS = True # Solo para desarrollo
```

Apps instaladas

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'rest_framework',  
    'corsheaders',  
    'tasks',  
    'inventario',  
    'colaboradores',  
    'proyectos',  
    'calendario',  
]
```

Email configuration

```
EMAIL_BACKEND = 'django.core.mail.backends.smtp.EmailBackend'  
EMAIL_HOST = 'smtp.gmail.com'  
EMAIL_PORT = 587  
EMAIL_USE_TLS = True
```

- **Instalación y Ejecución:**

```
cd backend  
pip install django djangorestframework django-cors-headers  
python manage.py migrate  
python manage.py runserver # Puerto 8000
```

ATLAS

Innovación al alcance de tus manos

5. MODELO DE DATOS



ATLAS

Innovación al alcance de tus manos



5.1 Entidades Principales

5.1.1 Usuario

- **PK:** id_usuario
- **Campos:**
 - **nombre:** Nombre del usuario.
 - **apellido:** Apellido del usuario.
 - **estado:** Estado del usuario (activo/inactivo).
 - **correoElectronico:** Correo electrónico del usuario.
 - **contraseña:** Contraseña del usuario.

ATLAS

Innovación al alcance de tus manos

- **suscripcion:** Tipo de suscripción del usuario.
- **terminoServicio:** Indica si el usuario aceptó los términos de servicio.
- **auth_user_id:** Relación con el sistema de autenticación.
- **Relaciones:**
 - 1:N con Proyecto (un usuario puede tener varios proyectos).
 - N:M con ColaboradorProyecto (un usuario puede colaborar en varios proyectos).

5.1.2 Proyecto

- **PK:** id_proyecto
- **Campos:**
 - ☐ **nombreproyecto:** Nombre del proyecto.
 - ☐ **fechacreacion:** Fecha de creación del proyecto.
 - ☐ **id_usuario:** Relación con el usuario que creó el proyecto.
- **Relaciones:**
 - N:1 con Usuario (un proyecto pertenece a un usuario).
 - N:M con ColaboradorProyecto (un proyecto puede tener varios colaboradores).

5.1.3 Cliente

- **PK:** dni
- **Campos:**
 - nombre: Nombre del cliente.
 - apellido: Apellido del cliente.
 - correo: Correo electrónico del cliente.
 - telefono: Teléfono del cliente.
 - proyecto: Relación con el proyecto al que pertenece.
- **Relaciones:**
 - 1:N con Agendamiento (un cliente puede tener varios agendamientos).

5.1.4 Tareas

- **PK:** id_Tarea
- **Campos:**
 - **nombreTarea:** Nombre de la tarea.
 - **fechaCreacion:** Fecha de creación de la tarea.
 - **fechaLimite:** Fecha límite para completar la tarea.
 - **descripcion:** Descripción de la tarea.
 - **filtro:** Estado de la tarea (por completar/completada).
 - **tipoTarea:** Tipo de tarea.
 - **id_usuario:** Relación con el usuario asignado.
 - **fechaActual:** Fecha de la última actualización.
- **Relaciones:**
 - N:1 con Usuario (una tarea pertenece a un usuario).

5.1.5 ColaboradorProyecto

- **PK:** id
- **Campos:**
 - **fecha_asignacion:** Fecha en que el colaborador fue asignado al proyecto.
 - **usuario_id:** Relación con el usuario colaborador.
 - **proyecto_id:** Relación con el proyecto.
- **Relaciones:**
 - N:1 con Usuario (un colaborador pertenece a un usuario).
 - N:1 con Proyecto (un colaborador pertenece a un proyecto).

5.1.6 Equipo

- **PK:** numeroSerie
- **Campos:**
 - **marca:** Marca del equipo.
 - **fotoEquipo:** URL de la foto del equipo.
- **Relaciones:**
 - 1:N con EquipoAgendamiento (un equipo puede estar relacionado con varios agendamientos).

5.1.7 Agendamiento

- **PK:** idAgendamiento
- **Campos:**
 - **Cliente_dni:** Relación con el cliente.
 - **Usuario_id:** Relación con el usuario que gestiona el agendamiento.
- **Relaciones:**
 - N:1 con Cliente (un agendamiento pertenece a un cliente).
 - N:1 con Usuario (un agendamiento pertenece a un usuario).

5.1.8 EquipoAgendamiento

- **PK:** id
- **Campos:**
 - **fechaIngreso:** Fecha de ingreso del equipo.
 - **fechaSalida:** Fecha de salida del equipo.
 - **comentarioEntrada:** Comentarios al ingresar el equipo.
 - **comentarioSalida:** Comentarios al salir el equipo.
 - **estado:** Estado del equipo.
 - **equipo_numeroSerie:** Relación con el equipo.
 - **agendamiento_idAgendamiento:** Relación con el agendamiento.
- **Relaciones:**
 - N:1 con Equipo (un equipo puede estar relacionado con varios agendamientos).
 - N:1 con Agendamiento (un agendamiento puede incluir varios equipos).

ATLAS

Innovación al alcance de tus manos

5.2 Diccionario de Datos

TABLA O ENTIDAD	ATRIBUTO O CAMPO	TIPO DE DATO	LONGITUD	LLAVE PRIMARIA	DESCRIPCION	TABLA O ENTIDAD FORANEA
Equipo	numeroSerie	Entero	12	PK	El numero de serie del Equipo	-
	marca	Cadena	20	-	la marca del equipo	-
	fotoEquipo	Cadena	-	-	Foto del estado del equipo	-
Cliente	dni	Entero	10	PK	Numero identificador del cliente	-
	nombre	Cadena	30	-	Nombre/s del cliente	-
	apellido	Cadena	30	-	Apellido/s del cliente	-
	telefono	Entero	10	-	Numero telefonico del cliente	-
	correo	VARCHAR	40	-	Correo del cliente	-
	proyecto	Entero	-	FK	Proyecto al cual esta asociado	Proyecto(id_proyecto)
Usuario	idUsuario	Entero	-	PK	Identificador único del usuario.	-
	nombre	Cadena	50	-	Nombre del usuario.	-
	apellido	Cadena	50	-	Apellido del usuario.	-
	estado	Enum	-	-	Estado del usuario (activo/inactivo).	-
	correoElectronico	Cadena	100	Única	Correo electrónico del usuario.	-
	contraseña	Cadena	100	-	Contraseña del usuario.	-
	suscripcion	Enum	-	-	Tipo de suscripción del usuario (gratuito/premium).	-
	terminoServicio	Booleano	-	-	Indica si el usuario aceptó los términos de servicio.	-
	auth_user_id	UUID	-	Única	Relación con el sistema de autenticación.	-
	rol_id	Entero	-	FK	Relación con el rol asignado al usuario.	Rol(idRol)
	token	Cadena	300	-	Token de autenticación del usuario.	-
	uuid_supabase	UUID	-	Única	Identificador único generado por Supabase.	-
	enlacePerfil	Texto	-	-	URL del perfil del usuario.	-
Rol	idRol	Entero	-	PK	identificador del rol	-
	nombre	Cadena	50	-	Nombre del rol (administrador/colaborador).	-
Tarea	id_Tarea	Entero	-	PK	Identificador único de la tarea.	-
	nombreTarea	Cadena	50	-	Nombre de la tarea.	-
	fechaCreacion	Date	-	-	Fecha en que se creó la tarea.	-
	fechaLimite	Time	-	-	Fecha límite para completar la tarea.	-
	descripcion	Cadena	300	-	Descripción detallada de la tarea.	-
	filtro	Enum	-	-	Estado de la tarea (por completar/completada).	-
	tipoTarea	Enum	-	-	Tipo de tarea (administrativa, técnica, etc.).	-
	id_usuario	Entero	-	FK	Relación con el usuario asignado a la tarea.	Usuario(idUsuario)
	fechaActual	Timestamp	-	-	Fecha de la última actualización de la tarea.	-
historial_colaboradores	id	Entero	-	PK	Identificador único del historial.	-
	usuario_id	Entero	-	FK	Relación con el usuario colaborador.	-
	proyecto_id	Entero	-	FK	Relación con el proyecto asociado.	Proyecto(id_proyecto)
usuarioTarea	estado	Cadena	20	-	Estado del colaborador en el proyecto (activo/eliminado).	-
	id_usuarioTarea	Entero	-	PK	Identificador único de la relación usuario-tarea.	-
	para_todos	Booleano	-	-	Indica si la tarea es visible para todos los usuarios.	-
	especifico	Booleano	-	-	Indica si la tarea es específica para un usuario.	-
	id_usuario	Entero	-	FK	Relación con el usuario asignado.	Usuario(idUsuario)
Agendamiento	id_tarea	Entero	-	FK	Relación con la tarea asignada.	Tareas(id_Tarea)
	id_Agendamiento	Entero	-	PK	Es un identificador único de cada registro de Agendamientos	-
	Cliente_dni	Entero	-	-	numero identificador del cliente en la tabla agendamiento	-
EquipoAgendamiento	Usuario_id	Entero	-	FK	numero identificador del usuario	usuario(Usuario_id)
	agendamiento_Equipo	Entero	-	PK	Identificador único de la relación entre equipo y agendamiento.	-
	fechaIngreso	-	-	-	Fecha en que el equipo ingresó al sistema.	-
	fechaSalida	Date	-	-	Fecha en que el equipo salió del sistema.	-
	comentarioEntrada	Cadena	300	-	Comentarios al ingresar el equipo.	-
	comentarioSalida	Cadena	300	-	Comentarios al salir el equipo.	-
	Estado	Enum	-	-	Estado actual del equipo (activo/inactivo).	-
	equipo_NumeroSerie	Cadena	12	FK	Relación con el equipo asociado.	Equipo(numeroSerie)
Proyecto	agendamiento_idAgendamiento	Entero	-	FK	Relación con el agendamiento asociado.	Agendamiento(idAgendamiento)
	fotoEquipo	Cadena	300	-	URL de la foto del equipo.	-
	id_proyecto	Entero	-	PK	Identificador único del proyecto.	-
	nombreproyecto	Cadena	50	-	Nombre del proyecto.	-
	fechaCreacion	Timestamp	-	-	Fecha en que se creó el proyecto.	-
	id_usuario	Entero	-	FK	Relación con el usuario que creó el proyecto.	Usuario(idUsuario)

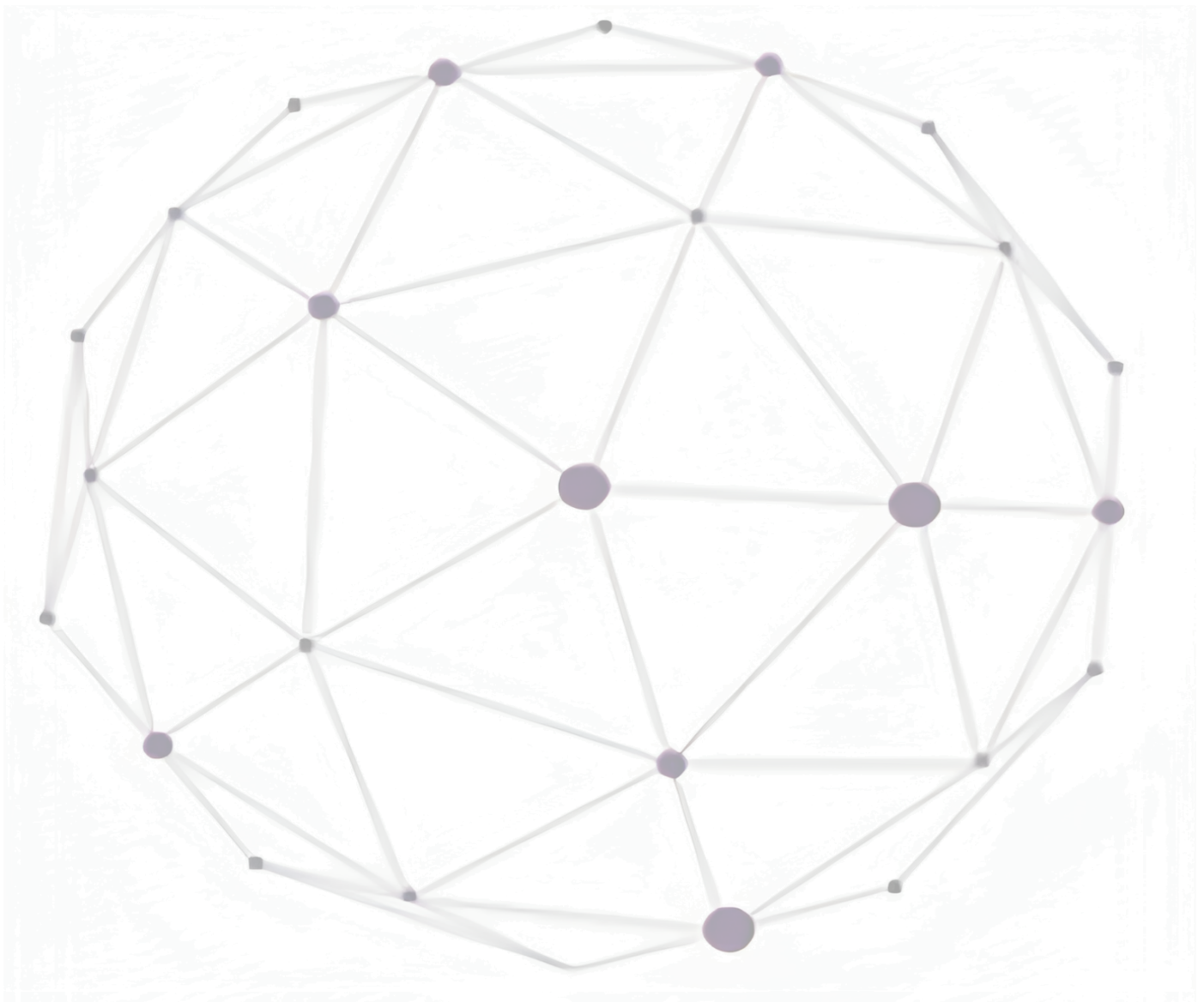
ATLAS

Innovación al alcance de tus manos

5.3 Diagrama de Clases

- Diagrama de Clases:

https://drive.google.com/file/d/1q8JzoTJwO6F5DIgmjSYJImUThHxNcUWD/view?usp=drive_link



6. APIs y Endpoints

6.1 Endpoints Criticos

- **Gestión de Proyectos**

POST /tasks/api/v1/Proyecto/

GET /tasks/api/v1/proyectos_colaboradores/?id_usuario={id}

GET /tasks/api/v1/info_proyecto_colaboradores/?id_proyecto={id}

- **Gestión de Colaboradores**

POST /tasks/api/v1/invitacionColaborador/

GET /tasks/api/v1/filtro_colaborador/?id_proyecto={id}

GET

/tasks/api/v1/estado_colaborador_proyecto/?id_usuario={id}&id_proyecto={id}

- **Patron de Response**

// Ejemplo de respuesta exitosa

```
{  
  "status": "success",  
  "data": {  
    "nombreproyecto": "Proyecto Demo",  
    "fechacreacion": "2025-01-15"  
  }  
}
```

NOTA: Todos los endpoints requieren autenticación via Supabase Auth headers.

ATLAS

Innovación al alcance de tus manos

7. AUTENTICACIÓN Y SEGURIDAD

7.1 Flujo de Autenticación

1. **Login** → Supabase Auth
2. **Obtener perfil** → Django API
3. **Validar acceso** → Hook (useProjectAcces)

7.2 Sistema de Roles

```
const ROLES = {  
  ADMIN: 1,    // Puede invitar colaboradores  
  COLABORADOR: 2 // Solo lectura del proyecto asignado  
};
```

7.3 Validación de Acceso

- **Archivo:** useProjectAccess.js
- **Lógica:** Verificar estado “Activo” en ColaboradorProyecto
- **Fallback:** Redireccionar a Error 404

NOTA: Supabase maneja automáticamente la validación de Tokens por medio de sus JWT tokens y refresh tokens

8. PATRONES DE DESARROLLO

8.1 Patron de Gestión de Información

- **Estrategia de Cache:** Zustand con //persist middleware
- **Actualizaciones Optimas:** Actualización inmediata + rollback en error
- **Sistema de Eventos:** Custom events para sincronización

8.2 Ejemplo de Patrón de Cache:

// useProjectsStore.js - Patrón Cache

```
fetchProjectInfo: async (projectId) => {  
  const { lastProjectId, projectName } = get();  
  if (lastProjectId === projectId && projectName) {  
    console.log("Proyecto ya en cache, no recargando...");  
    return;  
  }  
  // Cargar desde API...  
}
```

Component Patterns

- **HOC Pattern:** DataTable para tablas reutilizables
- **Compound Components:** Modales con FloatingModal
- **Custom Hooks:** Lógica compartida (useProjectAccess)

9. SOLUCIÓN DE PROBLEMAS

9.1 Problemas Comunes

- **Error: "Colaborator State Changed"**

- **Causa:** Event listener no registrado
- **Solución:** Verificar `window.addEventListener('collaboratorStateChanged')`
- **Archivo:** `SendCollaborations.jsx`

- **Error 404 en Dashboard**

- **Causa:** Usuario sin acceso activo
- **Solución:** Verificar estado en tabla ColaboradorProyecto en Supabase
- **Query:** `estado_colaborador_proyecto/?id_usuario={id}&id_proyecto={id}`

- **Cache No Actualiza**

- **Causa:** Zustand persist mantiene datos obsoletos
- **Solución:** Implementar `forceRefresh()` o limpiar `localStorage`

9.2 Logs de Debugging

// Habilitar logs en desarrollo

```
console.log("Proyecto cargado:", nombreProyecto);  
console.warn("No se pudo cargar el nombre del proyecto");
```

ATLAS

Innovación al alcance de tus manos

ARCHIVOS DE REFERENCIA CLAVE

FUNCIONALIDAD	ARCHIVO PRINCIPAL
Gestión Usuario	stores/useUserStore.js
Cache Proyectos	useProjectsStore.js
Validación Acceso	useProjectAccess.js
Tabla Reutilizable	DataTable.jsx
Email Templates	mensajeColabo.html
Configuración Supabase	supabase/client.js
Layout Principal	components/layout/DashboardLayout.jsx
Configuración Django	backend/settings.py
Views Principales	backend/tasks/views.py

CONTACTO TECNICO

- **Email:** contacto@atlascompany.com
- **Teléfono:** +57 301 243 3965
- **Documentación:** Este manual técnico
- **Código:** Revisar comentarios en archivos fuente

Este manual técnico documenta la arquitectura y funcionamiento actual de ATLAS SOFT v1.0. Para obtener la información más actualizada sobre nuevas funcionalidades o cambios en el sistema, consulte los comentarios en el código fuente o contacte al equipo de desarrollo

Desarrollado con pasión para microempresas de cómputo que buscan digitalizar y optimizar sus procesos de gestión.

© 2025 Atlas Company. Todos los derechos reservados.
